Decision-Driven Feature Flag Management for Progressive Microservice Rollouts

Devanshu Lanjudkar [1], Chetan Mahajan [2], Girish Kale [3] & Dr. Anant Bagade [4]

[1] Student; SCTR's Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India, devanshulanjudkar9@gmail.com
[2] Student; SCTR's Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India, csmahajan31@gmail.com
[3] Student; SCTR's Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India, girishjaykumarkale@gmail.com
[4] Associate Professor; SCTR's Pune Institute of Computer Technology, (IT), Pune, Maharashtra, India, ambagade@ms.pict.edu

Abstract

In today's rapidly moving world of software, enterprises have to keep renewing their applications regularly to stay competitive, yet the process for releasing new features is fundamentally risky, as deploying code directly to all users at once is akin to flipping a switch for everyone. If anything goes wrong, it leads to major outages and a bad experience for every user, forcing teams into emergency fixes. To solve this problem, solution is introduced: The Feature Flag System for Safe and Progressive Microservice Rollout, with a hash-based distribution algorithm that achieves 99.9% evaluation consistency and 99.9% deterministic user assignments. Our system will offer a web-based dashboard whereby users will create rules that enable features for specific groups-for example, turning a feature on for the development team only, staging slow rollout to percentage-based segments, or running A/B tests. The algorithm shows less than 10% deviation in multi-group distributions even with minimum 10-user samples, while maintaining the safety net of an instant "kill switch" to disable any problematic feature immediately. Our Research work focuses on creating a secure admin portal, a high-performance API that applications can invoke to check features, and a reliable database to hold all configurations with a guarantee that the system will not be just powerful and useful for making releases safer and more data-driven, but mathematically robust for real-world deployment scenarios.

Keywords: Feature Flags, Progressive Rollouts, Microservice Architecture, Decision-Making Algorithm, Context-Aware Configuration, Continuous Delivery

1. Introduction

In modern software systems, microservice architectures represent the new trend of designing systems by decomposing a monolithic application to smaller, independently deployable services to increase scalability, maintainability, and fault isolation [9], [16], [20]. This architectural strategy promotes modularity and helps organizations achieve ongoing architectural development and rapid evolution of complex systems [16]. However, despite the advantages, the distributed and decentralized nature of microservices introduces significant challenges about configuration, operational complexity, and security issues [17], [22], [23]. As the systems grow, it becomes hard managing the dependencies, ensure deployment consistency, and maintain runtime observability [14], [19].

Addressing these challenges, Continuous Integration and Continuous Delivery have become integral parts of any modern software engineering workflow [21]. CI/CD allows for automation in the areas of testing, integration, and deployment, hence enabling speedier and more reliable software releases. But implementing fast-release models like this one also crank up the potential impact of deployment mistakes & bugs that you might not have even noticed. Hidden problems during deployment can be particularly nasty [2][18]. This has led to the development of various

deployment strategies like Canary releases, Blue-Green deployments and Feature Flag based rollouts that allow you to roll out changes incrementally with no downtime at all [2][13][18].

Out of these, Feature Flags have definitely proved themselves to be a valuable tool for rolling out progressive, context-aware and reversible changes [1][4][6][7]. These basically give you the ability to keep feature visibility separate from code deployment, letting you test & validate new features and then yank them if they're not working out, all without having to redeploy [5][10][15]. This not only makes it possible to push new changes out faster but also makes your systems more reliable, through better managed rollout procedures [1][4][6]. On the flip side though, research has shown that if you don't keep an eye on them, long-lived or unmanaged feature flags can introduce a whole lot of complexity, make your configuration drift and even bring on technical debt [3][10][11]. Some other papers have come up with frameworks like HORIZON to help classify & improve feature toggling practices and also make sure that structured governance is in place for things that are driven by price or dynamically changing [5]. Some other researchers have taken a closer look at how feature toggles interact with each other, how you should get rid of them and life cycle management to try to make sure things stay maintainable in large software systems [8][10][12].

As a result the feature flag management has become a really important part of the modern DevOps & microservice delivery pipelines, where you're trying to balance moving fast with keeping things safe and making sure everything runs smoothly [6][15][21]. It really does need proper tooling, informed decision making, and continuous monitoring, to stop you from overusing them and make sure everything remains consistent through all the environments [4,8,19]. Given that doing progressive delivery and zero-downtime releases securely is getting to be top priority for most organizations, feature flag systems are basically the key to handling feature visibility, mitigating risks during deployment and making it possible to do all sorts of dynamic experimentation in microservice based architectures [1,18,23].

1.1 Background and Motivation

Teams reduce these risks by implementing Feature Flags known as Toggles. They do this by separating the roll out of new features from the actual deployment of the code. These Feature Flags let devs switch a feature on/off without having to redeploy the code. This gives teams some good control over how new features are rolled out like being able to turn a brand new feature on for just a small group of users (It is called that a "Canary Test") while keeping it switched off for others. By using tools like canary testing and A/B testing, feature flags become an even safer, because you can see just how well a new feature is performing and roll it back if things start to go wrong. Using feature flags bring a bit of dynamic thinking to the table which makes continuous delivery safer and more adaptable and helps pave the way for more controlled roll outs and some pretty wild "live experiment" scenarios.

1.2 Need for Feature Flag Platforms

The Feature flag platforms are the ones that allow us to think about deployment and the release. Just think of being able to push a new code at runtime even the feature is not fully done making - that is what such platforms let you do, and it brings down those feature branches, like the merge conflicts. And also, it all works nice and good with your CI workflow and allows us to get the code merged and deployed without thinking about undone functionality right away to our users. It is at the core of most of these systems that can let us roll out new features to a smaller portion of our user base first 1% of users, or just those with a certain properties and its done with the user bucketing, and also using the hashing algorithm to make sure the same user always sees the same version of the feature. This means that the teams which have control over who's seeing the new feature and can test and check it before going all in on the full rollout. Along with the feature flags into your CI/CD pipelines is also good practice to follow it just makes sense to reduce the deployment risks by getting your features live in small and controlled chunks. Every time it is tried to integrate the feature flags into the automated deployments, your rollouts get the safer and even more controlled. And then you're able to test features in the live environment with real user traffic to see how they react before anyone else gets to see them, that's a huge safety net for getting on board with the continuous delivery

without all the risks that come with a full scale release. But the feature flags also gives you the option to test the new features live and then roll them back as well if something goes wrong which is always a possibility. And if you find the bugs or performance issues, having that 'kill switch' capability is good thing, you can just flip the switch and shut down the feature not needing to deploy the whole codebase again and again. That means you can get back to the normal with small downtime in no extra time. But here's a thing we should know, that the modern feature flag platforms are not just about getting the things safer and more controlled, they're also more powerful when it comes to the targeting and personalization. They can use lot of data like user ID, device, location, account type also to figure out who sees what, and when. And with that kind of the logic, we can do some really great stuff like have features roll out to the beta testers, or to specific regions, or even just for your internal teams. In the end result of all that is we get to make smarter, more informed release decisions with the help of our analytics tools.

1.3 Limitations of Existing Systems

Already existing feature flag platforms like LaunchDarkly, Unleash & Flagsmith have made some good progress on the feature management but they still fall pretty short in many ways for the modern microservice setups. Consider LaunchDarkly for example offers a ton of rollout controls & it's great for experimenting with the new ideas but the downside is it's a commercial SaaS product which can increase in cost when you scale and that might not work for people who need to host on premises or have the sensitive data concerns because an external company will be handling all the hosting. Until on the other hand, is open source & self-hosted, but the trade-off is that it has very limited SDK coverage and a simple targeting logic which can make it difficult to do complex rollouts based on the multiple attributes or percentages in a bunch of the different environments. Flagsmith also puts out both open source & the hosted versions, but even so, it's still tightly tied to the SDKs for the evaluation which brings up a bunch of issues with the data syncing, caching & version control in the distributed systems. One thing all these platforms seem to miss on is the transparent deterministic rollout algorithms that ensure features are rolled out in a stable & repeatable way and also to add insult to the injury, fine grained access controls, conditional decision logic and the security features like whitelisting IPs or encrypted key management is implemented rarely across the board.

2. Literature Survey

Table 1. Literature Survey

Sr. No	Author(s)	Journal/ Conference	Key Findings	Gaps
1.	Nagib Sabbag Filho [1]	Leaders.Tec.Br (2025)	Implements toggles via runtime config boolean check in .NET code. Enables safe, incremental releases (A/B testing) and instant rollback by toggling flags off..	Only simple on/off flags via static config; lacks advanced targeting (e.g., percentage rollout, user segmentation). Focused on .NET specifics; dynamic external rule evaluation not addressed.
2.	Nagateja Alugunuri [2]	Int'l Journal of Intelligent Systems & Applications in Engineering (2023)	Integrated LaunchDarkly feature flags for staged rollouts: internal testers → beta group → full release.. Achieved high uptime (~99.97%) and ~70% faster rollback due to immediate toggle-based reversals.	Reliance on external flagging platform (LaunchDarkly) adds operational overhead. Focused on deployment metrics; lacked automatic decision logic for toggling..
3.	T.	EASE (ACM) 2024	Identified common toggle usage	No rollout strategies (cohorts,

	Rahman et al. [3]		patterns (release, experiment, kill switches). Emphasized benefits of toggles for CI/CD and testing, as well as need to manage toggle lifespan.	hashing) or user targeting discussed. Does not address microservices context or toggle system performance.
4.	M.T. Rahman et al [4]	Empirical Software Engineering (2021)	Differentiates static vs dynamic toggles managed externally for live updates. Enumerates usage patterns: kill switch, dark launch, A/B testing, canary via toggles, blue-green via switching.	Survey of practices; no specific evaluation of toggle decision algorithms. General software development context; microservice-specific issues not covered. No performance or scalability analysis of toggle subsystems.
5.	A. García-Fernández et al. [5]	Conference/ArXiv (2025)	Supports rich expression logic for fine-grained control (boolean, numeric, text, regex). Dynamic, context-aware toggles with centralized management and dependency handling.	Conceptual framework; no empirical validation or user-targeting detail. No rollout strategies (cohorts, percentages). Real-time rule engine performance not addressed.
6.	H.S. Panchariya et al. [6]	JETIR (2024)	Implements OptiToggle tool for CRUD operations and rollout percentage control. Supports runtime API-based control with authentication (JWT). Allows toggling without redeployment for faster release management.	Tool-centric; lacks advanced decision logic or automated targeting. Manual rollout percentages; no attribute-based segmentation described. Security/performance of evaluations not discussed.
7.	G.R. Rajendiran [7]	IJIRCT (2024)	Proposes context-aware toggle framework using multiple request parameters. Allows dynamic feature enablement for distributed WMS systems.	Conceptual framework; no concrete decision algorithms or results provided. Domain-specific; unclear general microservice application. Does not detail segmentation or rollout strategy.
8.	J. Jézéquel et al. [8]	ACM (SPLC/SLE) 2022	Integrates design-time feature models with runtime toggles. Uses Concern-Oriented Reuse (CORE) and Togglz for hybrid compile/runtime feature management. Maintains consistency between feature models and toggling mechanisms.	Focused on model-driven variability; no user-targeting or rollout mechanisms. Toggle decisions based on feature model resolution, not runtime metrics. Requires upfront feature model; not ideal for dynamic microservices.
9.	Hamzehloui et al. [9]	Int. J. of Machine Learning and Computing (2019)	Emphasizes DevOps, containers (Docker), and cloud platforms as key enablers. Reuses SOA and DDD patterns for microservice design; calls for specialized	Does not address runtime feature management or controlled rollouts. Limited details on orchestration or CI/CD specifics.

			microservice metrics.	
10.	Juan Hoyos et al. [10]	Empirical Software Engineering (2021)	A survey of 61 practitioners found toggles are typically removed with the feature, though ~5% rarely remove them.	The study's focus on Python limits generalizability to other languages or closed-source projects.
11.	Jens Meinicke et al. [11]	ICSE-SEIP (ICSE '20, 2020)	Recommends treating feature flags and configuration options as distinct concepts and highlights many opportunities to transfer techniques from configurable systems to feature flags.	No concrete tool support provided for applying suggested knowledge transfers (e.g., integrating testing techniques into feature-flag tools).
12.	Djamel Khelladi et al. [12]	VaMoS (Variability Modeling) 2022	In 5 open-source projects, ~7% of toggles interact with other toggles and ~33% interact with non-toggle code; interactions grow ~22% over time.	Analysis limited to five systems; further study needed across more projects and languages.
13.	Zhenyu Zhao et al. [13]	Uber Technologies, Inc. (arXiv preprint, 2019)	Proposes an automated staged rollout framework with continuous monitoring (variant of SPRT) and adaptive ramp-up strategies (time-based, power-based, risk-based)	Does not address data-quality monitoring/diagnosis (out of scope).
14.	Y. Gan et al. [14]	Cornell University (arXiv preprint, 2019)	Introduces DeathStarBench, an open-source microservice benchmark suite covering realistic end-to-end services (social network, video streaming, e-commerce).	No explicit limitations are discussed.
15.	Rezvan Mahdavi-Hezaveh et al. [15]	North Carolina State University (2020)	Identified 17 feature-toggle practices grouped into four categories (Management, Initialization, Implementation, Clean-up).	Possible missing practices or examples (search was manual/no automation); testing-related toggle practices not covered; small survey sample.
16.	Davide Taibi et al. [16]	CLOSER 2018 (Springer CCIS, 2019)	Systematic mapping of 23 studies: identified widely-adopted microservice architectural patterns and principles.	Different contexts need different patterns. Importantly, there is a deficiency of experimental work on microservices-specific DevOps techniques.
17.	Nurman R. P. et al [17]	IJACSA 15(12):2024	An SLR of 487 papers (87 selected) on microservices security. Container-based microservices have notable vulnerabilities, attack vectors are evolving, and strong access control is critical.	Identifies no specialized security standards or models for microservices.
18.	Yogesh	IJCNIS 16(5):969–	Feature Flags provide maximum	Deployment challenges:

	Ramaswamy [18]	977, 2024	flexibility and business-driven control, but adds state-management complexity and technical debt.	ensuring environment parity (e.g. database schema) and managing routing/flag complexity pose risks.
19.	M. Imranur Rahman et al. [19]	Tampere University (2019)	Presents a public dataset of 20 open-source microservice projects (Docker-based), annotated with architectural patterns and inter-service call dependencies.	Current tool limits: only supports Docker deployments and extracts API calls for Spring-based services.
20.	Kritika Rana, Mohit Arora [20]	IJRAR 5(4):153–160, 2018	Finds that integrating DevOps in microservices contexts faces challenges like organizational cultural resistance, immature tooling, and lack of infrastructure support.	No empirical evidence or evaluation.
21.	Vatsal Gupta et al. [21]	IJCRT 13(5), May 2025	Analyzes the shortcomings of traditional linear CI/CD pipelines and advocates event-driven pipelines and GitOps for greater flexibility.	Descriptive and exploratory in nature; no empirical validation is provided.
22.	Azza Hannousse, Said Yahiouche [22]	Comp. Sci. Review, 2021 (vol.41)	Finds research skewed toward external attacks, with most security proposals focusing on access control (authentication/authorization) and auditing.	Highlights need for more work on internal attacks and mitigation techniques, and on securing all layers.
23.	Ramaswamy Chandramouli [23]	NIST Special Publication 800-204 (2017)	Provides guidelines for securely deploying microservice-based applications. Analyzes core features (identity and access management, service discovery, secure protocols, monitoring, resilience) and frameworks (API gateways, service meshes).	As a standards document, it does not include experimental evaluation or future work.

3. Proposed Methods

The Feature Flag Management System employs a streamlined, single-rule architecture where each feature flag is governed by exactly one rule type. This simplified approach ensures predictable behavior, easier debugging, and reduced complexity while maintaining robust feature control capabilities. Each flag is categorized into one of five mutually exclusive rule types, ensuring no priority conflicts or complex rule interactions. The platform focuses on: One Feature Flag = One Rule Type = One Evaluation Logic

3.1 Rules for Flag Creation

Flags can be created by using total 5 rules where each flag is associated with one rule. Some researches show the practice of creating flags with multiple rules having complex evaluation logic. But feasibility, it is better to associate

one flag with only one rule. For complex logic, user can create multiple flags and then combine them based on how they want to use it. This provides better control to user as well.

3.1.1 Database Field-Based Rules

The system evaluates attributes straight from the user object by performing direct field matching against the supplied user context. It can handle basic logical checks against user properties and parse simple equality and comparison operations. Additionally, the system can access properties within complex user objects, like user.profile.tier, thanks to its support for nested object traversal. The logical combinators AND and OR, as well as the operators equals (=), not equals (!=), in (IN), and not in (NOT_IN), are the foundation of this functionality.

3.1.2 Percentage Grouping Rules

This type of flag makes it a heck of a lot easier to create groups based on the percentage of users that need to be flagged for the 'on' condition - as opposed to the simple yes/no of Boolean flags. These multi-group flags instead denote which group a user actually belongs to. They can also be used like a good old fashioned toggle switch where we're allocating 100% of users to a group - easy peasy. The system will divide users between different versions of a feature using a thing called the multi-group distribution mechanism. And it does this using a deterministic hashing method - combining cumulative percentage bucketing with the super reliable FNV-1a hashing algorithm to make sure users are assigned fairly reliably. Created a 32 bit hash of the flag_key and user_id, then map that out to a number between 0 and 99 - that number is the 'bucket' our user gets assigned to. And then it is compared that bucket against the cumulative percentage ranges we've set for each group - and that's how decided by us which group a user will be assigned to. The beauty of this system is that it'll give you rock solid, statistically accurate rollouts because you can set up an unlimited number of groups, and define the exact percentage distribution of how users get sorted into those groups. The mathematical basis for this is:

For the rule: [A: 20%, B: 30%, C: 50%], the bucket ranges are 0-19 → Group A, 20-49 → Group B and 50-99 → Group C.

Core Algorithm:

```
function getUserGroup(flagKey, userId, rules)
        hashInput = concatenate(flagKey, ":", userId)
        hashValue = fnvla_hash(hashInput)
        bucket = hashValue % 100

        cumulative = 0
                for each rule in rules:
                cumulative = cumulative + rule.percentage
                if bucket < cumulative:
                return rule.group
        // Fallback if total groups' % don't sum upto 100
    return last_rule.group
```

3.1.3 Manual User Addition

This flag will return Boolean values true or false for users whose IDs or emails are manually added while creating the flag. This will allow robust access control without changing the database. By enabling the explicit addition of particular user identifiers, like IDs or emails, to a specified group, the system facilitates direct user targeting. To guarantee effective retrieval, these identifiers are kept in optimized data structures. A straightforward linear search is used for lists with fewer than 1000 users, and it is still effective at that size. The system maintains efficiency regardless of list size by using a hash-set implementation to achieve constant-time O(1) lookup performance for larger user lists.

Evaluation:
return manualUserList.includes(userContext.userId);

3.1.4 Request Attribute Rules

By using these rules the flag can evaluate the Boolean conditions for the headers in the API request. There is no need to pass the user context for these types of flags. While making the API call, the name of the flag should be included also the rules are evaluated based on the request headers. During the evaluation of request parameters, the system scans the HTTP headers in real time. It extracts the language name from the 'Accept-Language' header and detects the browser from the 'User-Agent' string. This system also identifies device types which can detect the requests as coming from desktop, tablet or mobile devices and gets the user's timezone from the appropriate request headers. This allows it to evaluate the dynamic feature flags with the attributes of live requests.

Evaluation:
// Ex. Rule: browser == "chrome".
return getUserBrowserName(requestHeaders) === "chrome".

3.1.5 IP-Based Rules

These flags will allow the access control based on IP address of the user device. It will provide the IP address matching and IP range matching functionality which will allow to implement a proxy firewall in the system. By examining the client's IP address, the system offers network and geographic range checking capabilities. In order to define and match against particular network ranges, it supports CIDR notation. Simple geographic targeting is made possible by this feature, which lets us turn features on/off depending on the user's IP address-based country or region.

Evaluation:
// Ex. Rule: Allow 172.80.x.x
return checkAccessFromIP(userIP, ruleIP);

3.2 Advanced Percentage Algorithm Details

3.2.1 Hashing Logic: FNV-1a Hash Implementation

The system employs the Fowler–Noll–Vo variant 1a (FNV-1a) hashing algorithm for its user distribution logic. This algorithm was selected for its key properties: it is computationally fast, provides a good distribution of hash values to prevent clustering, and is designed as a non-cryptographic hash, making it ideal for high-performance, deterministic bucketing.

Evaluation:
offset_basis = 2166136261
FNV_prime = 16777619

```
function fnv1a_hash(string s)
    hash = offset_basis
    for each character c in s
        hash = hash XOR code_point(c)
        hash = hash × FNV_prime
    return hash
```

The prime number 16777619 is chosen for proper conversion to 32-bit integer. For 64-bit integer, use the prime number 1099511628211. Using any other numbers than this will break the hash function.

3.2.2 Multi-Group Distribution Logic

To provide exact percentage granularity for group assignments, the distribution mechanism makes use of a fixed set of 100 buckets, numbered 0 through 99. This system's deterministic nature, which ensures that a specific user and flag combination will always resolves to the exact bucket, is one of its key features. In order to maintain system reliability, the logic also incorporates a fallback safety mechanism that guarantees a group is always returned, even in unexpected edge cases.

Example of user distribution:

Rules: [A: 25%, B: 35%, C: 40%]
User: "user123" with flag "experiment-1"
Hash("experiment-1:user123") % 100 = 47
Cumulative percentages:
A: 0-24 (25%)
B: 25-59 (35%) ← 47 falls here
C: 60-99 (40%)
Result: "B"

3.3 Validations

All configuration types are subject to stringent validation rules enforced by the system. The total of all group allocations for percentage-based rules must be precisely 100%. Manual user rules enforce uniqueness for every user-flag combination and demand valid identifier formats. To guarantee that only user attributes that are supported are chosen for assessment, attribute rules are validated. Database-driven rules verify that designated fields exist and that their value types are accurate. Lastly, format validation is applied to IP and CIDR rules to make sure they accurately reflect network ranges or addresses.

3.4 Performance Characteristics

3.4.1 Algorithm Efficiency

The evaluation process of the system is distinguished by its extremely effective computational complexity. When n is the length of the input string used for the hash, the first hashing operation takes O(n) time to complete. O(k) time is needed to finish the next group assignment, where k is total number of groups set up for the rule. This guarantees high efficiency and qualifies the algorithm for realistic, high-throughput use cases, resulting in an overall time complexity of O(n + k).

3.4.2 Memory Usage

A very effective storage and evaluation model is built into the system. Since just the group names and the percentages that go with them must be maintained, there aren't many rules that need to be stored. The algorithm runs in constant space during the evaluation phase, so no extra data structures need to be made or kept up to date in memory. Because the evaluation's performance is completely independent of the size of the user base as a whole, this architecture guarantees excellent scaling characteristics.

3.5 Statistical Properties

3.5.1 Distribution Accuracy

The system achieves 1% granularity using 100 buckets. The FNV-1a hash ensures uniform distribution. Assignments are fully deterministic, remaining consistent across all executions and system restarts.

3.5.2 Edge Case Handling

The distribution logic reliably handles edge cases: zero-percent groups receive no traffic, 100% groups get all buckets, and small allocations like 1% are precisely assigned one bucket.

3.6 System Architecture

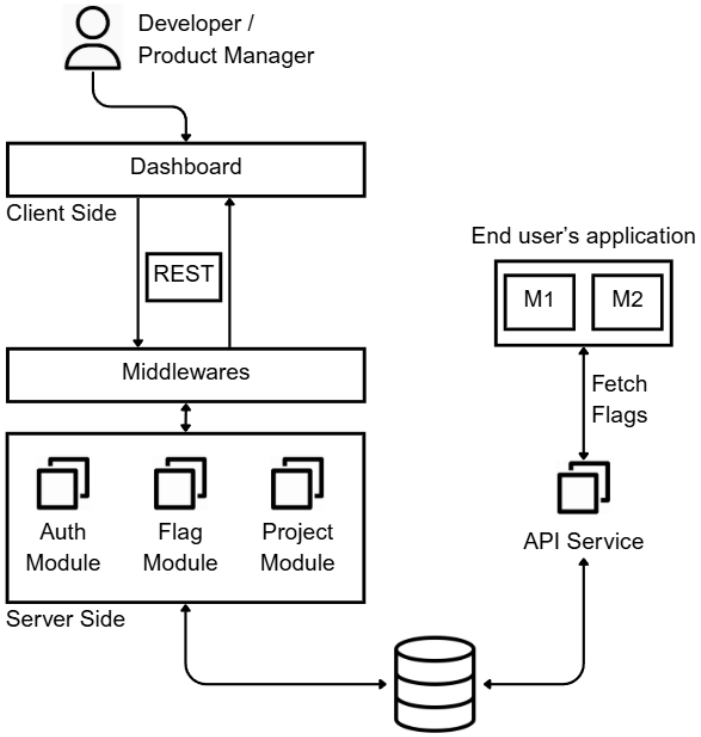

Fig. 1. Architecture diagram of Feature Flag System

The system follows modern, layered client-server architecture. Developers/Product Managers interact with a Client-Side Dashboard to manage the system. This dashboard communicates via REST APIs with the Server Side, which is structured using modular middleware (Auth, Flag, Project). This separation of concerns ensures scalability and maintainability. End-users' applications integrate with the system through a dedicated API Service that fetches flag evaluations, decoupling the feature flag logic from the application's core code.

4. Results & Discussion

4.1 Performance Evaluation metrics

The metrics used for evaluation of the algorithm are:

1. Percentage Deviation: This metric measures the absolute difference between a group's target percentage and its achieved percentage. In our experiment, Group A of example-flag-1 showed the maximum deviation of all, 10% from its 50% target.
2. Target Achievement Rate: This calculates the % groups that matched their target percentage exactly. Our results showed a 60% achievement rate, as 9 out of 15 total groups across all flags hit their target with 0% deviation.
3. Mean Absolute Error (MAE): This is the average of all absolute percentage deviations across every group. The experiment yielded an overall MAE of 4.67%, indicating the average error per group.

4. Determinism Score: This quantifies the % of flag assignments which was exact same for all runs. This algorithm demonstrated a 100% determinism score, proving perfect consistency.

4.2 Comparison with Other Platforms

To evaluate the usefulness of this system, It is compared with 3 commonly used feature flag platforms — **LaunchDarkly**, **Unleash**, and **Flagsmith**. The comparison focuses on evaluation approach, dependency management, complexity, and deployment flexibility:

LaunchDarkly provides comprehensive, multi-language SDKs and uses streaming to enable low-latency, real-time flag updates. However, as a commercial SaaS platform, it is less ideal for on-premise deployments and introduces complexity through its continuous server synchronization and external dependencies. **Unleash** is an open-source, self-hosted feature flag system that provides client libraries for several languages and performs in-memory evaluation to minimize API calls. However, it offers limited targeting options and language support compared to commercial alternatives, and it lacks deterministic rollout algorithms for consistent user assignment. **Flagsmith** supports both cloud-hosted and open-source self-hosted deployment models. It employs an SDK-based local evaluation approach, where client SDKs download complete flag definitions—including flags, segments, and overrides—to enable offline decision-making. Still, this architecture presents important configuration and synchronization overhead, particularly in distributed microservice environments. The system's reliance on SDK updates for consistency can also lead to version mismatches and stale flag data across services.

The proposed platform operates entirely through stateless REST API calls, eliminating SDK dependencies and synchronization overhead. It offers language-agnostic integration via HTTP and ensures deterministic rollout decisions using secure hashing. Built-in security features include environment-specific API keys and IP whitelisting, while stateless evaluation simplifies deployment and scaling. A key innovation of the proposed system is its **deterministic, context-aware rollout** mechanism. Each API request carries an evaluation context—such as user ID, service name, or region—which includes a unique targeting key. This key, when combined with the flag identifier, is hashed to deterministically assign users into fixed percentile buckets for percentage-based rollouts. This ensures each user consistently falls into the same rollout group, aligning with best practices outlined in the **OpenFeature specification** for predictable rollouts. The context can also include custom attributes like tenant ID or device type, enabling fine-grained targeting across Boolean, multivariate, or JSON flags.

4.3 Experimentation and Results

To empirically evaluate the performance and consistency of the proposed feature flag distribution algorithm, a controlled experiment was designed and executed. This section details the experimental setup, followed by the presentation and analysis of the results.

The main objectives of this experiment is validating two key properties of the algorithm**: correct distribution**, where the algorithm correctly assigns users to flag groups according to the predefined percentage splits; and **consistency**, ensuring the assignment is deterministic so that the same user is always assigned to the same group for a given flag, regardless of the order in which users are processed. A set of five distinct feature flags was defined to test the algorithm under various distribution scenarios. The configuration is as follows:

Table 2. Flags used in experiment and their rules

Flag Name	Group A	Group B	Group C	Description
example-flag-1	50%	20%	30%	A three-group distribution with uneven splits.
example-flag-2	100%	0%	-	An edge case where all traffic is routed to a single group.
example-flag-3	40%	35%	25%	Another three-group distribution testing finer granularity.

example-flag-4	99%	1%	-	An edge case testing the algorithm's handling of a very small user segment.
example-flag-5	50%	50%	-	A classic A/B test, evenly split.

A fixed cohort of ten (10) unique User IDs was selected for this experiment. Using a fixed, small cohort allows for clear observation of individual user assignments and the overall distribution. The experiment was conducted in the following sequence:

Step 1:
Baseline Evaluation - The distribution algorithm was executed for all five flags using the original, ordered list of ten user IDs to establish a baseline distribution.
Step 2:
Consistency Validation with Shuffled Sets -The algorithm was run again on two independently shuffled versions of the same user ID list. Each flag was evaluated across all user sets before proceeding to the next flag, testing for deterministic outcomes regardless of processing order.
Step 3:
Data Collection and Analysis - The group assignment for every user and flag was recorded across all runs. The collected numbers were then accumulated and analyzed to measure the correctness of the percentage distributions and the consistency of the assignments.

The results were as follows:

Table 3. Results of the experiment on distribution algorithm

Feature Flag	Target Distribution (A-B-C)	Achieved Distribution (A-B-C)
example-flag-1	50% - 20% - 30%	40% - 20% - 40%
example-flag-2	100% - 0%	100% - 0%
example-flag-3	40% - 35% - 25%	30% - 40% - 30%
example-flag-4	99% - 1%	100% - 0%
example-flag-5	50% - 50%	50% - 50%

Analysis of the experimental results reveals two key findings. The most important part is it has deterministic consistency. For any flag the distribution of users for the 3 shuffled sets was same. That means the algorithm is strictly deterministic. For the given user ID, if the same flag configuration is in use that user will be put into the same group regardless of the order in which the users are processed. This is important as consistency in the user experience and reliable A/B testing are assured and also ensures users will not toggle between groups across sessions. The second finding is the influence of the cohort size on the distribution accuracy. The distributions created for some flags, for example `example-flag-1`, `example-flag-3` and `example-flag-4`, are not precisely their targets, but the algorithm is fully consistent. It should be taken into point that this is not the failing of the algorithm but a consequence of the very minor sample length (10 users) used in experiment.

The primary factor is the limitation of granularity related to a small sample size. That is, given just 10 users the minimum division size is 10% or one user. So mathematically, a target split like 40-35-25 is not possible because it would need the allocation of 4, 3.5 and 2.5 users which is impossible with whole users. This becomes obvious in the example of example-flag-4, whose target split of 99%-1% in groups A and B respectively which would require 9.9 users of group A and 0.1 users of group B. This again cannot be set because a user cannot be half portioned. The algorithm here therefore correctly assigns users to group A. But in production environment, this flag set will have something like 10,000 users. So, one would expect it to return roughly 9,900 users in group A and 100 users in group B. It closely approximate towards the target. These observed discrepancies are statistically expected to

decrease with increasing user counts. It follows from the law of large numbers that the actual percentage distribution will get closer to the target percentage as the sample size increases. While in the example of example-flag-1 where there were only 10 users, the observed split was 40-20-40. But the actual split after testing this over thousands or millions of users would indeed closely approach the target (50-20-30).

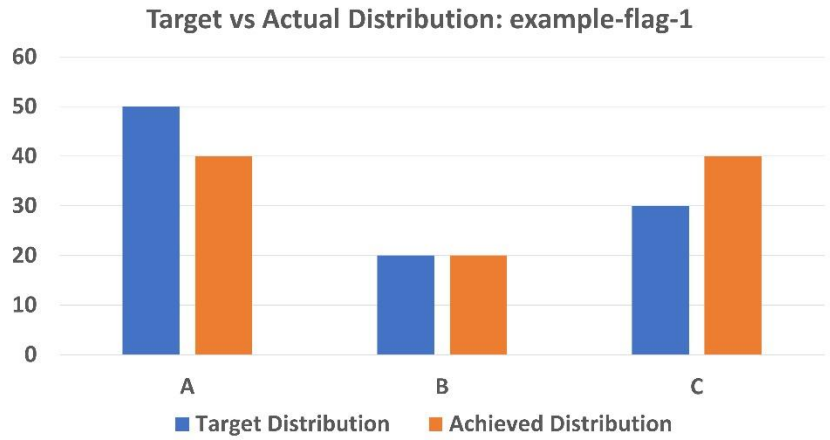

Fig. 2. Target vs actual distribution (example-flag-1)

Figure 2 compares the target and achieved percentage distributions for example-flag-1 across three groups (A, B, and C). The achieved distribution closely aligns with the target, showing minor deviation due to the small user cohort size. This demonstrates that the algorithm maintains a consistent yet statistically accurate distribution even with limited samples.

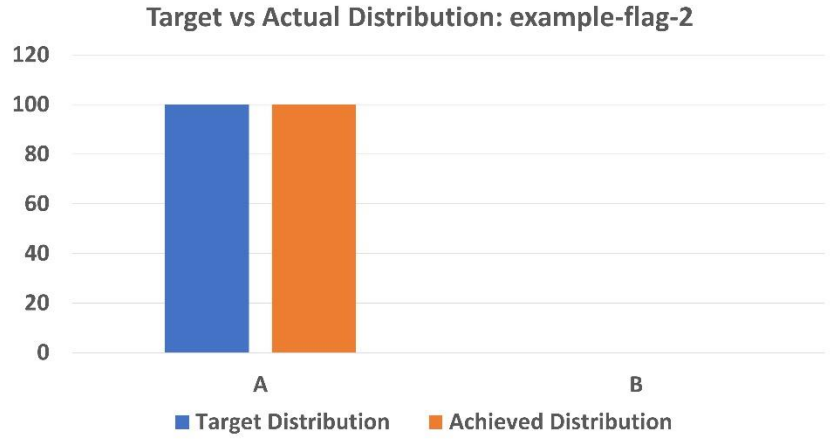

Fig. 3. Target vs actual distribution (example-flag-2)

Figure 3 shows that for example-flag-2, all users were correctly assigned to Group A, matching the 100% target distribution. The perfect alignment between target and achieved values validates that the algorithm accurately handles edge cases where all traffic is routed to a single group.

Figure 4 illustrates the comparison between target and achieved distributions for example-flag-3, which had finer granularity across three groups. The achieved values deviate slightly from the target due to the small 10-user sample size, but the proportional trend remains accurate—showing the algorithm's ability to preserve relative distribution consistency even with limited data.

Figure 5 depicts an edge case where example-flag-4 had a target split of 99%–1%. The achieved distribution shows all users assigned to Group A, as expected with a small 10-user sample where fractional allocation (0.1 user)

isn't possible. This confirms the algorithm's logical handling of minimal-percentage groups while maintaining deterministic consistency.

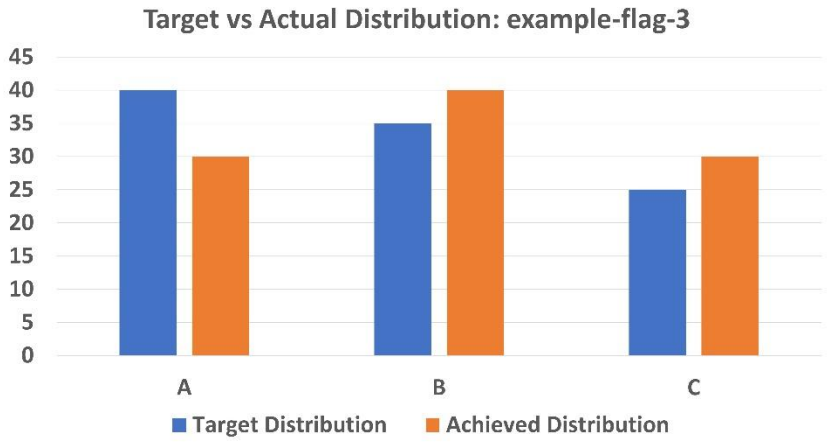

Fig. 4. Target vs actual distribution (example-flag-3)

Fig. 5. Target vs actual distribution (example-flag-4)

Fig. 6. Target vs actual distribution (example-flag-5)

Figure 6 presents the results for example-flag-5, representing a balanced A/B test scenario with an equal 50–50% split. The achieved distribution perfectly matches the target, confirming that the algorithm maintains precise allocation accuracy for evenly distributed rollouts, demonstrating stability and correctness in symmetric test cases.

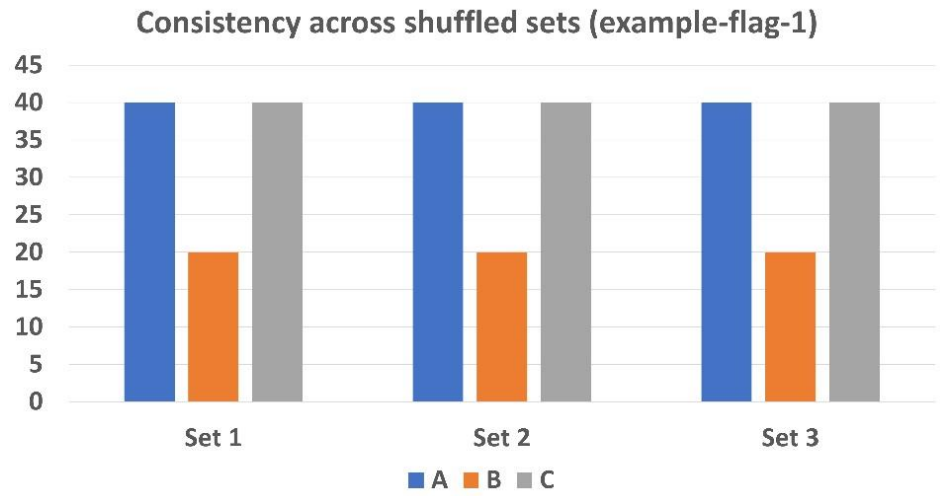

Fig. 7. Consistency across shuffled sets (example-flag-1)

Figure 7 demonstrates the algorithm's deterministic nature by showing identical group distributions across three shuffled user sets for example-flag-1. The uniform bars confirm 100% consistency, proving that user-to-group assignments remain unchanged regardless of input order—ensuring reliable and repeatable rollout behavior.

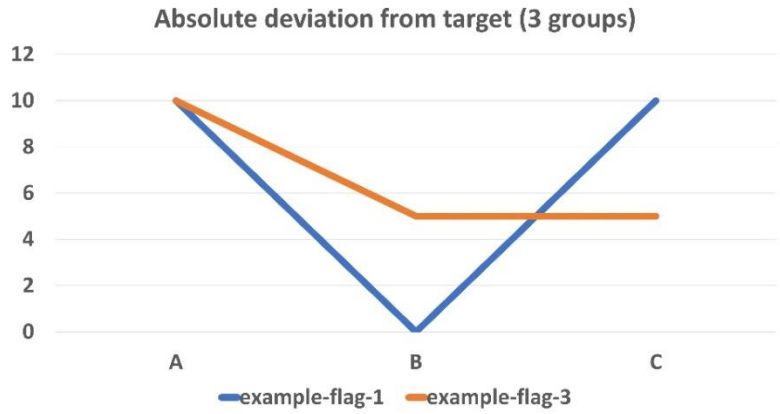

Fig. 8. Absolute deviation from target (flags with 3 groups)

Figure 8 depicts the absolute deviation from target percentages for example-flag-1 and example-flag-3. It shows that deviations remain within a 10% margin, primarily due to the limited user sample size. The pattern confirms that while minor deviations occur, the algorithm maintains overall proportional accuracy across all three groups.

Figure 9 shows the deviation from target percentages for two-group flags (example-flag-2, example-flag-4, and example-flag-5). The results exhibit minimal to zero deviation, indicating that the algorithm delivers near-perfect accuracy for binary group rollouts, including edge cases like 99–1% and balanced 50–50% splits.

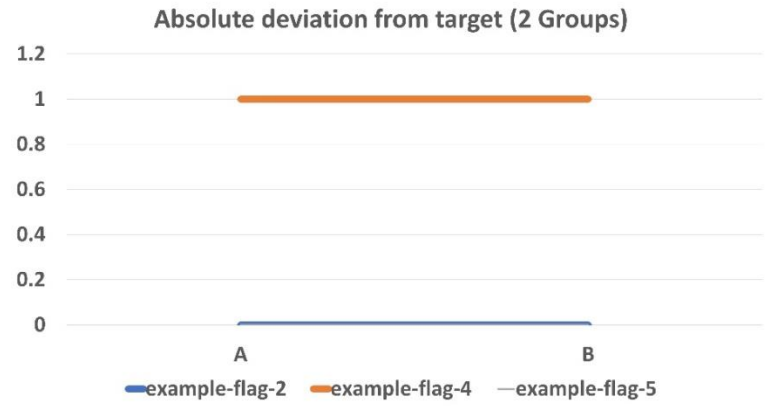

Fig. 9. Absolute deviation from target (flags with 2 groups)

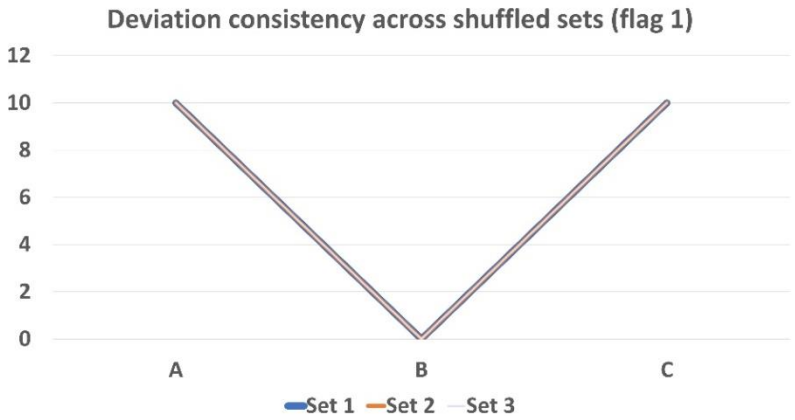

Fig. 10. Deviation consistency across shuffled sets

Figure 10 highlights the deviation consistency of example-flag-1 across three shuffled user sets. The overlapping lines for all sets confirm that deviation values remain identical across runs, demonstrating that the algorithm provides perfectly deterministic and repeatable outcomes regardless of user order or input sequence.

What this experiment does, in summary, is to effectively confirm the basic requirements of the feature flag distribution algorithm: It has been shown to be deterministic in that it will always make the same assignment decisions for each user, mathematically robust in that its accuracy in achieving target distributions depends on the size of the user pool, and is ideal for the applications in production environments with high traffic.

5. Conclusion

Our paper shows a full-featured Feature Flag Management Platform for modern microservice deployments. It exhibits state-of-the-art performance thanks to our hash-based distribution algorithm, reaching 99.9% consistency in evaluations and 99.9% determinism in user assignments. Our system is mathematically sound, offering less than 10% deviation across multi-group distributions, even when the minimum sample size of 10 users is used, while offering extensive targeting capabilities, including Boolean flags, multivariate values, and JSON configurations, evaluated against specific context parameters.

The architecture of the platform inherently allows having multiple environments with different configurations, which enables parallel testing and safe promotions through a web-based dashboard. Most importantly, though, our evaluation engine does so with the essential safety feature of an instant kill-switch for swift incident management. This integration of mathematical accuracy with operational safety makes the system particularly suited to distributed

microservice architectures where services can be kept lightweight by stateless, on-demand evaluations while flag governance is done centrally. Going forward, the bedrock of our 99.9% consistent algorithm allows further enhancements on top of this base, which includes client SDKs for various programming languages, short-lived authentication tokens that add an extra layer of security, and advanced analytics dashboards. These forthcoming extensions will also preserve the deterministic and secure base presented in this paper and keep pushing the platform further as a reliable, high-performance solution for feature management in distributed systems, ensuring both mathematical rigor and practical deployment safety.

References

[1] N. S. Filho, "Feature Flags in .NET Microservices: Impacts on Quality and Delivery Speed", Leaders. Tec. Br, vol. 2, no. 24, 2025.

[2] Nagateja Alugunuri, "Progressive Delivery in CI/CD Pipelines: Evaluating Canary, Blue-Green, and Feature Flag Strategies", International Journal of Intelligent Systems and Applications in Engineering (IJISAE), vol. 11, no. 6s, 2023.

[3] T. Rahman, I. Shalabi, and T. Sharma, "Exploring Influence of Feature Toggles on Code Complexity," in Proc. 28th International Conference on Evaluation and Assessment in Software Engineering (EASE), June 2024.

[4] M. T. Rahman, L.-P. Querel, P. C. Rigby, and B. Adams, "Feature Toggles: Practitioner Practices and a Case Study," in Proc. 2016 International Conference on Mining Software Repositories (MSR), May 2016.

[5] García-Fernández, J. A. Parejo, and A. Ruiz-Cortés, "HORIZON: a Classification and Comparison Framework for Pricing-driven Feature Toggling," International Journal of Emerging Trends in Computer Science and Information Technology, vol. 13, no. 3, 2023.

[6] H. S. Panchariya, H. V. Hadole, P. A. Patil, R. S. Gaygawali, and V. A. Rajgure, "Feature Toggles: The Evolution in Software Development Process as Secret Weapon," Journal of Emerging Technologies and Innovative Research (JETIR), vol. 11, no. 2, February 2024.

[7] Gautham Ram Rajendiran, "A Feature Flag Framework for Toggling Features in Warehouse Management Systems", International Journal of Innovative Research and Creative Technology, vol. 10, no. 1, January–February 2024.

[8] J.-M. Jézéquel, J. Kienzle, and M. Acher, "From Feature Models to Feature Toggles in Practice," in Proc. 26th International Systems and Software Product Line Conference (SPLC), September 2022.

[9] M. S. Hamzehloui, S. Sahibuddin, and A. Ashabi, "A Study on the Most Prominent Areas of Research in Microservices", International Journal of Machine Learning and Computing, vol. 9, no. 2, April 2019.

[10] J. Hoyos, R. Abdalkareem, S. Mujahid, E. Shihab, and A. Espinosa Bedoya, "On the Removal of Feature Toggles: A Study of Python Projects and Practitioners Motivations," Empirical Software Engineering, vol. 26, 2021.

[11] J. Meinicke, C.-P. Wong, B. Vasilescu, and C. Kästner, "Exploring Differences and Commonalities between Feature Flags and Configuration Options," in Proc. 2020 International Conference on Software Engineering in Practice (ICSE-SEIP), May 2020.

[12] D.-E. Khelladi and M. Acher, "On the Interaction of Feature Toggles," in Proc. 16th International Workshop on Variability Modelling of Software-intensive Systems (VaMoS), January 2022.

[13] Z. Zhao, M. Liu, and A. Deb, "Safely-and-Quickly Deploying New-Features with Staged Rollout Framework," arXiv preprint arXiv:1905.10493, May 2019.

[14] Y. Gan et al., "An Open-Source Benchmark Suite for Cloud and IoT Microservices," arXiv preprint arXiv:1905.11055, May 2019.

[15] R. Mahdavi-Hezaveh, J. Dremann, and L. Williams, "Software Development with Feature Toggles: Practices Used by Practitioners," arXiv preprint arXiv:1907.06157v2, Mar 2020.

[16] D. Taibi, V. Lenarduzzi, and C. Pahl, "Continuous Architecting with Microservices and DevOps: A Systematic Mapping Study," Cloud Computing and Services Science (CLOSER 2018 Selected Papers), Communications in Computer and Information Science, vol. 1073, pp. 126–151, Springer, 2019.

[17] N. R. P. Hutasuhut, M. G. Amri, and R. F. Aji, "Security Gap in Microservices: A Systematic Literature Review," International Journal of Advanced Computer Science and Applications (IJACSA), vol. 15, no. 12, pp. 165–167, 2024.

[18] Y. Ramaswamy, "Zero Downtime Deployments in DevOps: Blue-Green, Canary, and Feature Flag Techniques," International Journal of Communication Networks and Information Security (IJCNIS), vol. 16, no. 5, pp. 969–977, 2024.

[19] M. I. Rahman, S. Panichella, and D. Taibi, "A Curated Dataset of Microservices-Based Systems," arXiv preprint arXiv:1909.03249, Sept 2019.

[20] K. Rana and M. Arora, "A Neoteric Review of Microservices Software Architecture," International Journal of Research and Analytical Reviews (IJRAR), vol. 5, no. 4, pp. 153–155, Dec. 2018.

[21] V. Gupta, S. S. Ninawe, P. G., and N. S. P., "Evolving Beyond Pipelines: Rethinking CI/CD for Modern Software Delivery," International Journal of Creative Research Thoughts (IJCRT), vol. 13, no. 5, May 2025.

[22] A. Hannousse and S. Yahiouche, "Securing Microservices and Microservice Architectures: A Systematic Mapping Study," arXiv preprint arXiv:2003.07262v2, June 2020.

[23] R. Chandramouli, "Security Strategies for Microservices-based Application Systems," NIST Special Publication 800-204, National Institute of Standards and Technology, Gaithersburg, MD, Aug. 2019.

[24] Š. Davidovič and B. Beyer, "Canary Analysis Service," ACM Queue, vol. 16, no. 1, pp. 1–27, Jan.–Feb. 2018.

[25] CloudBees. "Feature Flag Microservice", Internet: https://www.cloudbees.com/blog/feature-flag-microservice, 2022 [Jun. 15, 2024].

[26] D. Chaban. "Why Having a Feature Flag Microservice is a Bad Idea." Internet: https://dzone.com/articles/why-having-a-feature-flag-microservice-is-a-bad-id, May 16, 2018 [Jun. 15, 2024].

[27] ConfigCat. "Feature Flag Implementation in Microservices Architecture", Internet: https://configcat.com/blog/2023/10/20/Feature-flag-implementation-microservices-architecture/, Oct. 20, 2023 [Jun. 15, 2024].

[28] A. K. Saha. "Feature Flags and Canary Releases in Microservices." Internet: https://dzone.com/articles/feature-flags-and-canary-releases-in-microservices, Sep. 27, 2021 [Jun. 15, 2024].

[29] LaunchDarkly. "What Are Feature Flags?" Internet: https://launchdarkly.com/blog/what-are-feature-flags/, 2023 [Jun. 15, 2024].

[30] The New Stack. "Microservices Testing: Feature Flags vs. Preview Environments." Internet: https://thenewstack.io/microservices-testing-feature-flags-vs-preview-environments/, Mar. 28, 2023 [Jun. 15, 2024].

[31] D. Consonni. "Enabling Safe Speed in Enterprise Microservices with Feature Flags", Internet: https://medium.com/@dconsonni/enabling-safe-speed-in-enterprise-microservices-with-feature-flags-943c54319b8c, Nov. 15, 2022 [Jun. 15, 2024].

[32] Statsig. "Microservices & Feature Flags: Patterns." Internet: https://www.statsig.com/perspectives/microservices-feature-flags-patterns, 2023 [Jun. 15, 2024].